

75 COMMON VIBE-CODED WEB APP VULNERABILITIES

— SECURITY HANDBOOK —



WHAT IT IS



WHAT CAUSES IT



HOW TO FIX IT



COPY-PASTE PREVENTION PROMPTS
FOR YOUR CODING LLM

A PRACTICAL DEFENSIVE GUIDE

How to use this handbook

This is a practical reference for modern web apps built with AI-assisted coding tools. These are 75 recurring vulnerability classes and security weaknesses worth checking for. It does not mean every app contains all 75.

Each entry explains the weakness, its usual cause, the potential impact, the defensive fix, and a prompt you can paste into a coding LLM while it has access to the relevant codebase.

Severity	Meaning
Critical	Could lead to major unauthorized access, account takeover, code execution, payment/security bypass, or broad data exposure.
High	Serious weakness with meaningful security or privacy impact.
Medium	Important weakness, usually with more limited impact or additional conditions.
Low	Defense-in-depth or reconnaissance-reduction issue that still matters in a mature app.

Important: an LLM prompt is not proof of security. High-risk controls such as authorization, authentication, payments, uploads, database policies, and webhooks still need to be verified with real tests.

VULNERABILITY 01 • HIGH

Broken Object Level Authorization (IDOR/BOLA)

WHAT IT IS

The app lets a logged-in user access another user's records by changing an ID or reference.

WHAT CAUSES IT

Authorization is checked only in the UI, or the server verifies login but not ownership/permission for the requested object.

POTENTIAL IMPACT

Attackers may read, change, or delete other users' private data.

HOW TO PREVENT / FIX IT

Enforce object-level authorization on every server-side request. Derive the user from the trusted session, check ownership/role before every read/write, use deny-by-default policies, and test cross-account access.

COPY-PASTE LLM PREVENTION PROMPT

> Review every endpoint and server action that reads, updates, or deletes a record. Add server-side object-level authorization so the authenticated user can access only records they own or are explicitly permitted to access. Do not trust IDs, user IDs, roles, or ownership values supplied by the client. Use the authenticated session as the source of identity. Add negative tests using two separate user accounts to prove cross-account access is denied. Do not weaken existing functionality.

VULNERABILITY 02 • CRITICAL

Broken Function Level Authorization

WHAT CAUSES IT

The frontend hides privileged buttons, but backend routes or server actions do not independently enforce roles/permissions.

POTENTIAL IMPACT

A normal account may perform administrative actions, change settings, manage users, or access restricted functions.

HOW TO PREVENT / FIX IT

Require server-side authorization for every privileged route/action. Centralize role/permission checks and default to deny.

COPY-PASTE LLM PREVENTION PROMPT

> Audit all privileged routes, API endpoints, server actions, RPC functions, and admin features. Enforce authorization on the server for every sensitive action using the authenticated user's trusted role/permissions. Never rely on hidden UI elements or client-side checks. Default to deny and add tests proving a normal user cannot call admin functions directly.

VULNERABILITY 03 • CRITICAL

Missing Authentication on Sensitive Endpoints

WHAT IT IS

Sensitive endpoints work without proving who the caller is.

WHAT CAUSES IT

Developers expose internal routes, APIs, server actions, or functions without authentication middleware.

POTENTIAL IMPACT

Unauthenticated attackers may access data or perform protected actions.

HOW TO PREVENT / FIX IT

Require authentication on all non-public endpoints and explicitly document the small set of routes intended to be public.

COPY-PASTE LLM PREVENTION PROMPT

> Inventory all routes, APIs, server actions, functions, and RPC endpoints. Mark each as public or authenticated. Add server-side authentication to every endpoint that handles user data or sensitive actions. Fail closed when the session is missing or invalid. Add automated tests confirming protected endpoints return an authorization error to unauthenticated requests.

VULNERABILITY 04 • CRITICAL

Privilege Escalation

WHAT IT IS

A lower-privileged user can gain permissions intended for admins or other trusted roles.

WHAT CAUSES IT

Roles or permission fields can be changed by the client, mass-assigned, or insufficiently validated.

POTENTIAL IMPACT

An attacker may become an administrator or gain powerful account capabilities.

HOW TO PREVENT / FIX IT

Keep authorization data server-controlled. Restrict role changes to explicitly authorized admin workflows and validate transitions.

created or updated. Make these fields server-controlled and reject client attempts to set them unless an explicitly authorized admin workflow permits it. Add tests proving ordinary users cannot promote themselves or other accounts.

VULNERABILITY 05 • CRITICAL

Insecure Direct Database Access from Client

WHAT IT IS

The browser talks directly to database operations that are broader than the user's actual permissions.

WHAT CAUSES IT

Database policies such as RLS are missing, incomplete, or bypassed while public client credentials can reach sensitive tables.

POTENTIAL IMPACT

Users may query or modify data outside their intended scope.

HOW TO PREVENT / FIX IT

Use strict database authorization policies, least-privilege client access, server-side operations for sensitive actions, and test policies with multiple identities.

COPY-PASTE LLM PREVENTION PROMPT

```
> Audit every database table, view, storage bucket, RPC, and client-accessible query. For systems such as Supabase, enable and verify row-level security where appropriate. Create explicit least-privilege policies for SELECT, INSERT, UPDATE, and DELETE. Move highly privileged operations to trusted server code. Test with two normal accounts and an unauthenticated client to confirm isolation.
```

VULNERABILITY 06 • CRITICAL

Exposed API Keys or Secrets in Frontend

WHAT IT IS

Private credentials are shipped in browser JavaScript, HTML, source maps, or public configuration.

WHAT CAUSES IT

Secrets are placed in client environment variables or hard-coded into frontend code.

POTENTIAL IMPACT

Anyone can extract and abuse the credential, potentially causing data access, charges, or account compromise.

HOW TO PREVENT / FIX IT

Keep secrets server-side only, rotate exposed keys, use restricted credentials, and proxy privileged third-party calls through trusted backend code.

COPY-PASTE LLM PREVENTION PROMPT

```
> Scan the frontend source, built assets, environment configuration, source maps, and repository for secrets. Remove all private API keys, service-role keys, database credentials, signing secrets, and private tokens from client-delivered code. Move privileged calls to server-side code, rotate any exposed credential, and use least-privilege scoped keys. Do not treat public client identifiers as secrets.
```

VULNERABILITY 07 • CRITICAL

Passwords, tokens, signing keys, or credentials are embedded directly in application code or committed files.

WHAT CAUSES IT

Developers hard-code secrets for convenience or commit local environment files.

POTENTIAL IMPACT

Repository access, logs, builds, or leaks can expose credentials.

HOW TO PREVENT / FIX IT

Store secrets in a proper secret manager or protected environment variables, remove them from history where necessary, and rotate compromised values.

COPY-PASTE LLM PREVENTION PROMPT

> Search the codebase and configuration for hard-coded passwords, tokens, API secrets, private keys, database URLs with credentials, and signing secrets. Replace them with protected server-side environment variables or a secret manager. Ensure secret files are excluded from version control. Rotate any credential that may have been committed or exposed.

VULNERABILITY 08 • CRITICAL

SQL Injection

WHAT IT IS

Untrusted input changes the meaning of a SQL query.

WHAT CAUSES IT

Queries are built by concatenating user input instead of using parameterized statements or a safe ORM.

POTENTIAL IMPACT

Attackers may read, alter, or delete database data and sometimes bypass authentication.

HOW TO PREVENT / FIX IT

Use parameterized queries/prepared statements, safe ORM APIs, strict input handling, and least-privilege database accounts.

COPY-PASTE LLM PREVENTION PROMPT

> Review every database query that includes user-controlled input. Replace string concatenation and dynamic SQL with parameterized queries, prepared statements, or safe ORM query builders. Where dynamic identifiers are unavoidable, use strict allowlists. Add injection-focused tests and ensure the application's database account has only the permissions it requires.

VULNERABILITY 09 • HIGH

NoSQL Injection

WHAT IT IS

User input changes the structure or operators of a NoSQL query.

WHAT CAUSES IT

Raw request objects or attacker-controlled operators are passed directly into database filters.

POTENTIAL IMPACT

Attackers may bypass filters, authentication, or access unintended records.

COPY-PASTE LLM PREVENTION PROMPT

> Audit all NoSQL queries and filters. Never pass raw request bodies, query objects, or user-controlled operators directly to the database. Construct filters from explicit allowed fields, validate expected scalar types, reject unexpected keys/operators, and add tests for operator injection and authentication/filter bypass.

VULNERABILITY 10 • CRITICAL

OS Command Injection

WHAT IT IS

User input becomes part of an operating-system command.

WHAT CAUSES IT

The app uses shell commands with concatenated filenames, URLs, options, or other user-controlled values.

POTENTIAL IMPACT

Attackers may execute commands on the server.

HOW TO PREVENT / FIX IT

Avoid shell execution when possible. Use safe library APIs; otherwise use argument arrays, strict allowlists, and a low-privilege runtime.

COPY-PASTE LLM PREVENTION PROMPT

> Find all uses of shell execution, process spawning, system commands, and command-line utilities. Replace them with native library APIs where possible. If a process must be invoked, pass arguments as a structured array without a shell, strictly allowlist acceptable values, and run with minimum privileges. Add tests using hostile metacharacters to confirm they are treated only as data.

VULNERABILITY 11 • CRITICAL

Server-Side Template Injection

WHAT IT IS

Attacker-controlled text is interpreted as template code on the server.

WHAT CAUSES IT

User input is inserted into a server-side template in an executable/template-expression context.

POTENTIAL IMPACT

Depending on the engine, attackers may access server data or execute dangerous operations.

HOW TO PREVENT / FIX IT

Never evaluate user content as template source. Use fixed templates and pass untrusted values only as escaped data.

COPY-PASTE LLM PREVENTION PROMPT

> Review server-side template rendering. Ensure user-controlled content is never compiled, evaluated, or treated as template source or expressions. Use fixed templates and pass user values only as data with the engine's escaping enabled. Remove unsafe evaluation helpers and add tests confirming template syntax submitted by users is rendered as text.

WHAT IT IS

Malicious script-like content is saved by the app and later rendered to other users.

WHAT CAUSES IT

Stored user content is inserted into HTML without context-appropriate escaping or safe sanitization.

POTENTIAL IMPACT

Visitors may have actions performed in their browser context or sensitive information exposed.

HOW TO PREVENT / FIX IT

Prefer output encoding. Sanitize only where rich HTML is intentionally supported, use a proven sanitizer, and enforce a strong CSP as defense in depth.

COPY-PASTE LLM PREVENTION PROMPT

> Identify all stored user-generated content such as comments, profiles, posts, messages, filenames, and admin-visible fields. Render plain text using framework-safe escaping. If rich HTML is required, sanitize it with a maintained allowlist-based library before rendering. Avoid unsafe HTML injection APIs and add stored-XSS regression tests. Add a restrictive Content Security Policy as defense in depth.

VULNERABILITY 13 • HIGH

Cross-Site Scripting (Reflected XSS)

WHAT IT IS

Malicious input from a request is immediately reflected into a page in an executable context.

WHAT CAUSES IT

Query parameters, errors, search terms, or other request data are rendered without safe encoding.

POTENTIAL IMPACT

A crafted link can cause script execution in a victim's browser.

HOW TO PREVENT / FIX IT

Use context-aware output encoding, safe framework rendering, and avoid unsafe HTML sinks.

COPY-PASTE LLM PREVENTION PROMPT

> Trace all URL parameters, form values, error messages, and other request data that are reflected into HTML. Ensure they are rendered through the framework's escaped text mechanisms and never inserted with unsafe HTML APIs. Encode for the correct output context and add regression tests using HTML/script-like payload strings.

VULNERABILITY 14 • HIGH

DOM-Based XSS

WHAT IT IS

Client-side JavaScript takes attacker-controlled data and writes it into a dangerous DOM sink.

WHAT CAUSES IT

Values from URLs, storage, messages, or APIs flow into innerHTML or similar unsafe browser APIs.

POTENTIAL IMPACT

Malicious links or data may trigger script execution in the browser.

COPY-PASTE LLM PREVENTION PROMPT

> Audit client-side data flows from location/search/hash, local storage, postMessage, API responses, and user input into DOM sinks. Replace innerHTML, outerHTML, document.write, and equivalent unsafe insertion with.textContent or safe framework rendering. Sanitize any intentionally supported HTML and validate postMessage origins.

VULNERABILITY 15 • HIGH

Cross-Site Request Forgery (CSRF)

WHAT IT IS

A user's browser can be tricked into sending an authenticated state-changing request from another site.

WHAT CAUSES IT

Cookie-authenticated state changes lack CSRF defenses and accept cross-site requests.

POTENTIAL IMPACT

Victims may unknowingly change account data or perform sensitive actions.

HOW TO PREVENT / FIX IT

Use SameSite cookies, CSRF tokens where needed, Origin/Referer validation, and never use GET for state changes.

COPY-PASTE LLM PREVENTION PROMPT

> Review every state-changing request that relies on browser cookies for authentication. Use SameSite=Lax or Strict where compatible, implement robust CSRF tokens for requests that need cross-site cookie behavior, validate Origin for sensitive actions, and ensure GET/HEAD never change state. Add cross-site request tests.

VULNERABILITY 16 • CRITICAL

Weak Password Storage

WHAT IT IS

Passwords are stored in a form that is too easy to recover if the database leaks.

WHAT CAUSES IT

Plaintext, reversible encryption, fast hashes, or weak password-hashing parameters are used.

POTENTIAL IMPACT

A database breach can expose users' passwords and reused credentials.

HOW TO PREVENT / FIX IT

Use a modern password hashing function such as Argon2id or a well-configured bcrypt/scrypt implementation with unique salts.

COPY-PASTE LLM PREVENTION PROMPT

> Inspect password storage and authentication code. Ensure passwords are never stored plaintext or with reversible encryption. Use a maintained password-hashing library with Argon2id where supported, or a properly configured adaptive alternative such as bcrypt. Use per-password salts, safe parameters, constant-time verification through the library, and provide a migration path for legacy hashes.

VULNERABILITY 17 • MEDIUM

The app makes account takeover easier by allowing very weak credentials or failing to defend against common credential abuse.

WHAT CAUSES IT

Poor password controls, no breached-password screening, and weak login protections.

POTENTIAL IMPACT

Attackers have an easier time taking over accounts with guessed or reused credentials.

HOW TO PREVENT / FIX IT

Allow long passphrases, block known compromised passwords where practical, support password managers, and combine with rate limiting/MFA.

COPY-PASTE LLM PREVENTION PROMPT

> Review password creation and reset rules. Support long passphrases and password managers, reject known compromised passwords where a reputable privacy-preserving service/library is available, avoid arbitrary composition rules that reduce usability, and combine the policy with login rate limiting and optional/required MFA for sensitive accounts.

VULNERABILITY 18 • HIGH

Missing Multi-Factor Authentication for Sensitive Accounts

WHAT IT IS

High-impact accounts rely on a single authentication factor.

WHAT CAUSES IT

Admin, financial, developer, or other sensitive accounts have no MFA option or requirement.

POTENTIAL IMPACT

A stolen password alone may be enough for account takeover.

HOW TO PREVENT / FIX IT

Support phishing-resistant MFA where possible and require MFA for privileged accounts.

COPY-PASTE LLM PREVENTION PROMPT

> Add MFA support for privileged and high-risk accounts using a maintained authentication provider/library. Prefer phishing-resistant methods such as passkeys/WebAuthn where practical. Require MFA for administrative roles, secure enrollment and recovery, and ensure MFA checks are enforced server-side rather than only in the UI.

VULNERABILITY 19 • HIGH

Session Fixation

WHAT IT IS

An attacker can cause a victim to authenticate using a session identifier the attacker already knows.

WHAT CAUSES IT

The session ID is not regenerated after login or privilege changes.

POTENTIAL IMPACT

The attacker may reuse the known session to access the authenticated account.

COPY-PASTE LLM PREVENTION PROMPT

> Review session lifecycle handling. Regenerate or rotate the session identifier immediately after successful authentication, MFA completion, privilege elevation, and other trust-level changes. Invalidate obsolete session IDs and add tests proving a pre-login session token cannot be reused as the authenticated session.

VULNERABILITY 20 • HIGH

Session Hijacking Exposure

WHAT IT IS

Session tokens are easier to steal or reuse than they should be.

WHAT CAUSES IT

Tokens are exposed to JavaScript unnecessarily, sent insecurely, logged, leaked in URLs, or have excessive lifetime.

POTENTIAL IMPACT

A stolen token may allow account takeover.

HOW TO PREVENT / FIX IT

Use Secure, HttpOnly, SameSite cookies where appropriate, TLS, short reasonable lifetimes, rotation, revocation, and no tokens in URLs/logs.

COPY-PASTE LLM PREVENTION PROMPT

> Audit how session tokens are created, stored, transmitted, logged, refreshed, and revoked. For browser sessions, prefer Secure, HttpOnly, appropriately SameSite cookies. Never place session tokens in URLs or logs. Use reasonable expiry, rotate tokens after sensitive events, support logout/revocation, and require HTTPS.

VULNERABILITY 21 • CRITICAL

JWT Signature Verification Errors

WHAT IT IS

The application accepts a JWT without correctly verifying that it was signed by a trusted issuer.

WHAT CAUSES IT

Signature verification is disabled, algorithms are misconfigured, or untrusted keys/claims are accepted.

POTENTIAL IMPACT

Attackers may forge identities or privileges.

HOW TO PREVENT / FIX IT

Use a maintained JWT library, pin expected algorithms, verify signature, issuer, audience, expiry, and required claims.

COPY-PASTE LLM PREVENTION PROMPT

> Audit every JWT verification path. Use a maintained library and require cryptographic signature verification. Explicitly allow only the intended algorithm(s), trusted signing key(s), issuer, audience, expiration, and required claims. Reject unsigned tokens, unexpected algorithms, invalid keys, expired tokens, and malformed claims. Add negative verification tests.

VULNERABILITY 22 • HIGH

The server treats token claims as sufficient permission without applying current application authorization rules.

WHAT CAUSES IT

Roles, tenant IDs, or permissions in tokens are stale, overly trusted, or accepted from untrusted sources.

POTENTIAL IMPACT

Users may retain or gain access they should not have.

HOW TO PREVENT / FIX IT

Verify trusted claims and still apply server-side authorization for the requested resource/action; keep sensitive authorization state current.

COPY-PASTE LLM PREVENTION PROMPT

> Review how JWT claims such as role, user ID, organization ID, and permissions are used. Verify claims only from a trusted issuer, then enforce current server-side authorization for each requested resource/action. Do not let client-provided or unverified claims control access. Define how role changes and revocation take effect.

VULNERABILITY 23 • HIGH

OAuth Redirect URI Weakness

WHAT IT IS

An OAuth flow can redirect authorization results to an attacker-controlled or unintended destination.

WHAT CAUSES IT

Redirect URIs are matched loosely, dynamically accepted, or built from user input.

POTENTIAL IMPACT

Authorization codes or tokens may be exposed to an attacker-controlled endpoint.

HOW TO PREVENT / FIX IT

Use exact registered redirect URIs and strict provider configuration; never accept arbitrary redirect destinations.

COPY-PASTE LLM PREVENTION PROMPT

> Audit OAuth/OIDC redirect handling. Use an exact allowlist of registered redirect URIs and require exact matching. Never construct callback destinations from untrusted input or accept arbitrary domains/subdomains. Validate state and PKCE where applicable, and add tests proving modified redirect_uri values are rejected.

VULNERABILITY 24 • HIGH

OAuth State/PKCE Validation Missing

WHAT IT IS

OAuth login lacks protections that bind the authorization response to the initiating browser/client.

WHAT CAUSES IT

The app omits or incorrectly validates state and, for applicable public clients, PKCE.

POTENTIAL IMPACT

Login CSRF, authorization response mix-ups, or code interception risks increase.

HOW TO PREVENT / FIX IT

> Review OAuth/OIDC login flows. Generate a cryptographically random state value per authorization attempt, bind it to the initiating session, verify it exactly once on callback, and reject missing/mismatched values. For applicable clients, implement PKCE using S256 and verify the code verifier correctly. Prefer the official provider SDK/library.

VULNERABILITY 25 • MEDIUM

Account Enumeration

WHAT IT IS

The app reveals whether a username, email, or account exists.

WHAT CAUSES IT

Login, registration, password reset, or invitation responses differ in text, status, or timing.

POTENTIAL IMPACT

Attackers can build lists of valid accounts for targeted attacks.

HOW TO PREVENT / FIX IT

Use consistent public responses and reasonable timing while still logging detailed internal outcomes.

COPY-PASTE LLM PREVENTION PROMPT

> Review login, signup, password reset, invitation, and account recovery responses. Make public messages and status behavior consistent whether an account exists or not, without breaking legitimate UX. Avoid exposing existence through API error details. Keep precise reasons only in protected server logs and add enumeration tests.

VULNERABILITY 26 • HIGH

Brute-Force Login Protection Missing

WHAT IT IS

Attackers can make large numbers of authentication attempts without meaningful resistance.

WHAT CAUSES IT

No rate limiting, progressive delays, anomaly controls, or MFA protection exists.

POTENTIAL IMPACT

Passwords are easier to guess or test at scale.

HOW TO PREVENT / FIX IT

Rate-limit by multiple signals, add progressive delays/risk controls, protect against distributed abuse, and avoid permanent lockouts that enable denial of service.

COPY-PASTE LLM PREVENTION PROMPT

> Add server-side abuse protection to authentication endpoints. Rate-limit login attempts using a combination of account and network/device signals, introduce progressive delays or risk-based challenges, and return non-enumerating errors. Avoid simple permanent account lockouts that attackers can abuse for denial of service. Log and alert on sustained attacks.

WHAT IT IS

Password reset tokens can be guessed, reused, leaked, or remain valid too long.

WHAT CAUSES IT

Tokens have low entropy, are stored insecurely, are not single-use, or are not invalidated after reset.

POTENTIAL IMPACT

Attackers may reset another user's password.

HOW TO PREVENT / FIX IT

Use cryptographically random, short-lived, single-use tokens, store only safe representations, and invalidate sessions as appropriate after reset.

COPY-PASTE LLM PREVENTION PROMPT

> Audit the password-reset flow. Generate reset tokens with a cryptographically secure random generator and sufficient entropy. Make them short-lived and single-use, store a hashed representation where practical, invalidate them immediately after successful use, prevent account enumeration, and invalidate or review existing sessions after a password change.

VULNERABILITY 28 • HIGH

Email Verification Bypass

WHAT IT IS

A user can obtain verified-account privileges without actually proving control of the email address.

WHAT CAUSES IT

Verification state can be client-controlled, tokens are weak/reusable, or protected features do not check verified status.

POTENTIAL IMPACT

Attackers may create trusted-looking or privileged accounts using addresses they do not control.

HOW TO PREVENT / FIX IT

Keep verification state server-controlled and use secure, expiring, single-use verification tokens.

COPY-PASTE LLM PREVENTION PROMPT

> Review email verification end to end. Keep the verified flag server-controlled, use cryptographically random expiring single-use verification tokens, bind them to the intended account/address, and enforce verified-email requirements on the server for features that depend on verification. Add tests for token reuse and client-side flag tampering.

VULNERABILITY 29 • HIGH

Insecure 'Remember Me' Tokens

WHAT IT IS

Long-lived login tokens can be stolen, predicted, or reused indefinitely.

WHAT CAUSES IT

Persistent tokens are weak, stored plaintext server-side, never rotated, or not revocable.

POTENTIAL IMPACT

A stolen persistent token may provide long-term account access.

COPY-PASTE LLM PREVENTION PROMPT

> Audit persistent login or 'remember me' functionality. Use cryptographically random high-entropy tokens, store only hashed token values server-side where practical, rotate tokens after successful use, set reasonable expiration, revoke them on logout/password/security changes, and never expose them in URLs or logs.

VULNERABILITY 30 • HIGH

Insecure Cookie Attributes

WHAT IT IS

Authentication or sensitive cookies lack browser protections.

WHAT CAUSES IT

Secure, HttpOnly, or appropriate SameSite attributes are missing or cookie scope is unnecessarily broad.

POTENTIAL IMPACT

Cookies are more exposed to interception, scripts, or cross-site requests.

HOW TO PREVENT / FIX IT

Set Secure and HttpOnly for session cookies, choose appropriate SameSite, minimize Domain/Path scope, and use HTTPS.

COPY-PASTE LLM PREVENTION PROMPT

> Audit all cookies. For authentication/session cookies, set Secure and HttpOnly, choose the strictest compatible SameSite value, and minimize Domain and Path scope. Do not store sensitive secrets in script-readable cookies unless the design explicitly requires it. Ensure production uses HTTPS and add automated cookie-attribute checks.

VULNERABILITY 31 • HIGH

CORS Misconfiguration

WHAT IT IS

The server allows untrusted websites to make browser requests that should be restricted.

WHAT CAUSES IT

Origins are reflected blindly, wildcards are used with sensitive data, or credentials are allowed too broadly.

POTENTIAL IMPACT

A malicious site may interact with authenticated APIs or read data when browser credentials are included.

HOW TO PREVENT / FIX IT

Use a strict allowlist of trusted origins, never reflect arbitrary origins, and enable credentials only where required.

COPY-PASTE LLM PREVENTION PROMPT

> Audit CORS configuration on every API. Define an explicit allowlist of trusted origins, methods, and headers. Never blindly reflect the Origin header. Do not combine permissive origins with credentialed requests. Enable Access-Control-Allow-Credentials only where required and add tests proving untrusted origins cannot read protected responses.

VULNERABILITY 32 • MEDIUM

The browser has little defense in depth against injected scripts or unsafe content sources.

WHAT CAUSES IT

CSP is absent or uses overly broad sources such as unsafe-inline/unsafe-eval without necessity.

POTENTIAL IMPACT

XSS and third-party script compromise can have greater impact.

HOW TO PREVENT / FIX IT

Deploy a restrictive CSP tailored to the app, preferably using nonces/hashes for scripts, and test in report-only mode before enforcement.

COPY-PASTE LLM PREVENTION PROMPT

> Create or tighten the application's Content-Security-Policy. Restrict default-src and explicitly define script-src, style-src, img-src, connect-src, frame-ancestors, object-src, and base-uri as needed. Prefer nonces or hashes over unsafe-inline, avoid unsafe-eval, and remove unnecessary third-party origins. Roll out with Report-Only first if needed, then enforce.

VULNERABILITY 33 • MEDIUM

Clickjacking

WHAT IT IS

An attacker can embed the app in a frame and trick a user into clicking hidden or disguised controls.

WHAT CAUSES IT

The application does not restrict framing.

POTENTIAL IMPACT

Users may unknowingly trigger sensitive actions.

HOW TO PREVENT / FIX IT

Use CSP frame-ancestors and, for legacy support where needed, X-Frame-Options.

COPY-PASTE LLM PREVENTION PROMPT

> Prevent unauthorized framing of sensitive pages. Add a Content-Security-Policy frame-ancestors directive allowing only required origins, normally 'none' or 'self'. Add X-Frame-Options for legacy compatibility where appropriate. Verify legitimate embedding use cases before restricting and test that untrusted sites cannot frame the app.

VULNERABILITY 34 • MEDIUM

Open Redirect

WHAT IT IS

The application redirects users to attacker-chosen external URLs.

WHAT CAUSES IT

Redirect destinations are taken directly from query parameters or user input.

POTENTIAL IMPACT

Attackers can abuse trusted links for phishing or authorization-flow problems.

COPY-PASTE LLM PREVENTION PROMPT

> Find all redirect, returnUrl, next, callback, and continue parameters. Prefer internal relative paths. If external redirects are genuinely required, validate destinations against an exact allowlist of trusted schemes and hosts. Reject protocol-relative URLs, encoded bypasses, and unexpected schemes. Add redirect validation tests.

VULNERABILITY 35 • CRITICAL

Server-Side Request Forgery (SSRF)

WHAT IT IS

The server fetches a URL chosen by a user, allowing access to destinations the user could not reach directly.

WHAT CAUSES IT

URL fetchers, webhooks, image importers, previews, or integrations accept arbitrary destinations without network restrictions.

POTENTIAL IMPACT

Attackers may reach internal services, cloud metadata, or sensitive network resources.

HOW TO PREVENT / FIX IT

Allowlist destinations where possible, block private/link-local/loopback ranges after DNS resolution, restrict protocols, control redirects, and isolate outbound networking.

COPY-PASTE LLM PREVENTION PROMPT

> Audit every server-side URL fetch feature, including imports, previews, webhooks, callbacks, image fetchers, and integrations. Prefer explicit destination allowlists. Otherwise restrict schemes to required protocols, resolve and block private/loopback/link-local/internal addresses, re-check after redirects/DNS resolution, set timeouts/size limits, and use outbound network controls. Add SSRF regression tests.

VULNERABILITY 36 • HIGH

Path Traversal

WHAT IT IS

User input manipulates a file path to access files outside the intended directory.

WHAT CAUSES IT

Filenames or paths are concatenated without canonicalization and containment checks.

POTENTIAL IMPACT

Attackers may read or overwrite sensitive files.

HOW TO PREVENT / FIX IT

Use generated identifiers, safe storage APIs, canonicalize paths, and verify the final path remains inside the allowed base directory.

COPY-PASTE LLM PREVENTION PROMPT

> Audit all file read/write/download paths. Avoid using user input as filesystem paths; map public IDs to server-controlled filenames instead. Where paths are necessary, canonicalize the final path and verify it remains within the intended base directory. Reject absolute paths and traversal sequences and add regression tests.

WHAT IT IS

The app loads local files or modules based on user-controlled values.

WHAT CAUSES IT

Dynamic include/template/file-loading functions receive untrusted paths.

POTENTIAL IMPACT

Sensitive files may be disclosed or unsafe content executed depending on the runtime.

HOW TO PREVENT / FIX IT

Use fixed mappings/allowlists instead of user-controlled paths and remove dynamic include behavior.

COPY-PASTE LLM PREVENTION PROMPT

> Find dynamic file inclusion, template selection, module loading, and file-serving logic. Replace user-controlled filesystem references with fixed server-side mappings or strict allowlists. Ensure resolved files stay inside approved directories and cannot select executable/configuration files. Add negative tests for traversal and unexpected file types.

VULNERABILITY 38 • CRITICAL

Unrestricted File Upload

WHAT IT IS

Users can upload files that the application treats as executable or otherwise dangerous.

WHAT CAUSES IT

The server trusts extensions/MIME types, stores uploads in executable locations, or lacks content and size restrictions.

POTENTIAL IMPACT

Uploads may lead to malware hosting, code execution, XSS, storage abuse, or data exposure.

HOW TO PREVENT / FIX IT

Allow only necessary file types, validate content, rename files, store outside executable web roots, limit size, and serve with safe headers.

COPY-PASTE LLM PREVENTION PROMPT

> Harden every file-upload feature. Define an allowlist of required file types, verify actual content rather than trusting extension or client MIME type, generate server-side filenames, enforce size/count limits, store uploads outside executable application paths, and serve them with safe Content-Type and Content-Disposition headers. Add malware scanning where risk warrants it.

VULNERABILITY 39 • HIGH

File Upload Content-Type Confusion

WHAT IT IS

A file is validated or served as one type but interpreted by the browser or downstream system as another.

WHAT CAUSES IT

The app trusts MIME headers/extensions or allows browser content sniffing.

POTENTIAL IMPACT

validate actual file format, set explicit response Content-Type, use nosniff, and force download when active content is not intended.

COPY-PASTE LLM PREVENTION PROMPT

> Audit uploaded-file validation and serving. Detect the actual permitted file format using trusted parsing/magic checks, not only extensions or client MIME values. When serving files, set an explicit safe Content-Type, X-Content-Type-Options: nosniff, and Content-Disposition: attachment for content that should not render inline.

VULNERABILITY 40 • HIGH

Insecure File Permissions / Public Storage

WHAT IT IS

Sensitive files are accessible to users or the public because storage permissions are too broad.

WHAT CAUSES IT

Cloud buckets, object storage, directories, or generated links default to public or have weak policies.

POTENTIAL IMPACT

Private documents, videos, backups, or user uploads may leak.

HOW TO PREVENT / FIX IT

Make storage private by default, authorize every access, use short-lived signed URLs where appropriate, and audit bucket policies.

COPY-PASTE LLM PREVENTION PROMPT

> Audit every storage bucket, object container, upload directory, and file-delivery path. Make sensitive storage private by default. Enforce access based on the authenticated user's permissions and use short-lived signed URLs only when needed. Remove public listing/read policies that are not intentional and test cross-user file access.

VULNERABILITY 41 • HIGH

Mass Assignment

WHAT IT IS

Attackers can set fields that the UI never intended them to control.

WHAT CAUSES IT

The backend binds entire request objects directly to database models or update operations.

POTENTIAL IMPACT

Users may change roles, ownership, prices, status flags, balances, or other protected fields.

HOW TO PREVENT / FIX IT

Explicitly allowlist writable fields for each operation and keep sensitive fields server-controlled.

COPY-PASTE LLM PREVENTION PROMPT

controlled. Add tests for unexpected protected fields.

VULNERABILITY 42 • HIGH

Parameter Tampering

WHAT IT IS

The server trusts values from the browser that should be calculated or controlled server-side.

WHAT CAUSES IT

Prices, discounts, user IDs, plan names, permissions, quantities, or workflow states are accepted directly from client requests.

POTENTIAL IMPACT

Attackers may manipulate transactions or application state.

HOW TO PREVENT / FIX IT

Treat client values as untrusted; derive security- and money-sensitive values from server-side records and validated rules.

COPY-PASTE LLM PREVENTION PROMPT

> Identify client-supplied values that affect authorization, ownership, pricing, discounts, entitlements, account state, or workflow transitions. Recalculate or retrieve these values from trusted server-side data instead of trusting the request. Validate allowed transitions and add tampering tests that modify hidden fields and API payloads.

VULNERABILITY 43 • HIGH

Business Logic Abuse

WHAT IT IS

A user follows technically valid requests in an unintended sequence to gain an unfair or unauthorized result.

WHAT CAUSES IT

Security is implemented per endpoint but not across the full business workflow.

POTENTIAL IMPACT

Attackers may bypass limits, duplicate benefits, skip required steps, or manipulate transactions.

HOW TO PREVENT / FIX IT

Model allowed workflow states and invariants server-side, validate transitions, and test abuse cases rather than only malformed input.

COPY-PASTE LLM PREVENTION PROMPT

> Map the application's critical workflows as explicit states and invariants. For purchases, subscriptions, invitations, approvals, credits, bookings, rewards, and similar flows, enforce allowed transitions on the server and make important operations idempotent. Add abuse-case tests for skipping steps, repeating actions, reordering requests, and manipulating state.

VULNERABILITY 44 • HIGH

Race Condition / Double Submission

WHAT IT IS

Critical operations lack transactions, locking, idempotency, or unique constraints.

POTENTIAL IMPACT

Users may receive duplicate credits, orders, bookings, withdrawals, or other unintended results.

HOW TO PREVENT / FIX IT

Use database transactions/constraints, atomic operations, idempotency keys, and concurrency-safe state changes.

COPY-PASTE LLM PREVENTION PROMPT

> Review money, inventory, booking, quota, invitation, credit, and other one-time operations for concurrency issues. Use atomic database operations, transactions, unique constraints, or appropriate locking. Add idempotency keys for retryable requests and test simultaneous duplicate submissions to prove the invariant holds.

VULNERABILITY 45 • HIGH

Missing Rate Limiting on Sensitive APIs

WHAT IT IS

An endpoint can be called excessively without controls.

WHAT CAUSES IT

No server-side quotas exist for expensive, sensitive, or abuse-prone operations.

POTENTIAL IMPACT

Attackers may brute force, scrape, spam, cause excessive costs, or degrade service.

HOW TO PREVENT / FIX IT

Apply endpoint-specific rate limits and quotas using appropriate identity/network signals and return safe retry behavior.

COPY-PASTE LLM PREVENTION PROMPT

> Identify abuse-prone endpoints such as authentication, password reset, OTP, invitations, search, AI generation, exports, uploads, messaging, and expensive API calls. Add server-side rate limits and quotas appropriate to each action, using account plus network/device signals where useful. Return 429 responses safely and log sustained abuse.

VULNERABILITY 46 • HIGH

Resource Exhaustion / Unbounded Requests

WHAT IT IS

A user can force the app to consume excessive CPU, memory, bandwidth, storage, or third-party spend.

WHAT CAUSES IT

Requests lack limits on payload size, pagination, query complexity, processing time, uploads, or generated output.

POTENTIAL IMPACT

The service may become slow, unavailable, or unexpectedly expensive.

HOW TO PREVENT / FIX IT

Set strict practical limits, timeouts, pagination caps, concurrency controls, and budgets for expensive operations.

COPY-PASTE LLM PREVENTION PROMPT

VULNERABILITY 47 • HIGH

GraphQL Excessive Query Complexity

WHAT IT IS

Clients can submit GraphQL queries that consume excessive resources or expose more data than intended.

WHAT CAUSES IT

Depth, complexity, batching, introspection, field authorization, or pagination are poorly controlled.

POTENTIAL IMPACT

Attackers may cause denial of service or retrieve excessive data.

HOW TO PREVENT / FIX IT

Enforce field-level authorization, depth/complexity limits, pagination, batching controls, and production-appropriate introspection settings.

COPY-PASTE LLM PREVENTION PROMPT

> If GraphQL is used, audit field-level authorization and query cost. Add maximum depth/complexity controls, pagination caps, batching limits, and timeouts. Ensure sensitive fields have server-side authorization independent of parent objects. Restrict introspection in production only when appropriate to the deployment, without treating it as the primary security control.

VULNERABILITY 48 • HIGH

Excessive Data Exposure

WHAT IT IS

APIs return more fields than the client actually needs.

WHAT CAUSES IT

Entire database records or sensitive internal fields are serialized by default.

POTENTIAL IMPACT

Private data, internal identifiers, secrets, or metadata may leak.

HOW TO PREVENT / FIX IT

Use explicit response schemas/DTOs and return only the fields required for the feature.

COPY-PASTE LLM PREVENTION PROMPT

> Review API and server-action responses. Replace raw database-record serialization with explicit response schemas or DTOs that include only fields required by the client. Remove secrets, internal flags, sensitive personal data, password hashes, tokens, and unnecessary metadata. Add response-shape tests to prevent accidental reintroduction.

VULNERABILITY 49 • HIGH

Sensitive Data in URLs

WHAT IT IS

Tokens, emails, personal data, or credentials are passed through GET parameters.

POTENTIAL IMPACT

URLs may be stored in browser history, logs, analytics, referrers, screenshots, and proxies.

HOW TO PREVENT / FIX IT

Keep sensitive values out of URLs; use request bodies or secure cookies/headers as appropriate and redact logging.

COPY-PASTE LLM PREVENTION PROMPT

> Search routes, query parameters, redirects, and links for tokens, passwords, session identifiers, private personal data, and other secrets. Move sensitive values out of URLs into secure request bodies, cookies, or headers as appropriate. Ensure logs and analytics do not capture sensitive parameters and rotate any exposed credentials.

VULNERABILITY 50 • HIGH

Sensitive Data in Logs

WHAT IT IS

Logs contain secrets or personal information that should not be recorded.

WHAT CAUSES IT

The app logs full request bodies, headers, tokens, passwords, payment data, or sensitive user records.

POTENTIAL IMPACT

Anyone with log access may gain credentials or private data.

HOW TO PREVENT / FIX IT

Use structured logging with explicit redaction/allowlists and minimize sensitive data retention.

COPY-PASTE LLM PREVENTION PROMPT

> Audit application, infrastructure, analytics, and error logs for secrets and sensitive data. Never log passwords, session tokens, authorization headers, reset tokens, private keys, or full payment credentials. Implement centralized structured logging with field-level redaction or allowlists, and minimize personal data. Add tests for log redaction.

VULNERABILITY 51 • MEDIUM

Verbose Error Messages

WHAT IT IS

Production errors reveal internal details useful to attackers.

WHAT CAUSES IT

Stack traces, SQL errors, file paths, environment details, or framework debug pages are returned to users.

POTENTIAL IMPACT

Attackers gain information about the application's internals.

HOW TO PREVENT / FIX IT

Return generic public errors with correlation IDs while recording detailed diagnostics only in protected logs.

COPY-PASTE LLM PREVENTION PROMPT

debug pages in production.

VULNERABILITY 52 • HIGH

Debug Mode Enabled in Production

WHAT IT IS

Development features expose internals or unsafe controls in the live application.

WHAT CAUSES IT

Debug flags, developer consoles, test routes, profilers, or verbose framework modes remain enabled.

POTENTIAL IMPACT

Sensitive information or powerful debugging capabilities may be exposed.

HOW TO PREVENT / FIX IT

Use environment-specific configuration and fail production builds/deployments if debug features are enabled.

COPY-PASTE LLM PREVENTION PROMPT

> Audit production configuration for debug mode, developer consoles, test endpoints, profilers, mock authentication, seed routes, verbose exception pages, and development-only tools. Disable or remove them in production. Add deployment checks that fail when known debug flags or test routes are enabled.

VULNERABILITY 53 • MEDIUM

Security Misconfigured HTTP Headers

WHAT IT IS

Important browser security controls are missing or incorrectly configured.

WHAT CAUSES IT

Headers such as HSTS, CSP, nosniff, frame protections, and referrer policy are absent or overly permissive.

POTENTIAL IMPACT

Browser-based attacks or information leakage may be easier.

HOW TO PREVENT / FIX IT

Set appropriate security headers centrally and test them in production-like environments.

COPY-PASTE LLM PREVENTION PROMPT

> Add a centralized security-header policy appropriate to the application. Review Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, frame-ancestors/X-Frame-Options, and Permissions-Policy. Do not copy a generic policy blindly; preserve required functionality and add automated header checks.

VULNERABILITY 54 • HIGH

TLS / HTTPS Misconfiguration

WHAT IT IS

Sensitive traffic is not consistently protected in transit.

POTENTIAL IMPACT

Credentials and private data may be exposed or modified in transit.

HOW TO PREVENT / FIX IT

Require HTTPS, use modern TLS configuration, redirect HTTP safely, and enable HSTS after confirming HTTPS coverage.

COPY-PASTE LLM PREVENTION PROMPT

> Review production transport security. Ensure all authenticated and sensitive traffic uses HTTPS, HTTP redirects to HTTPS without exposing secrets, certificates are valid and automatically renewed, and obsolete TLS/protocol settings are disabled by the hosting platform. Enable HSTS once all relevant subdomains are confirmed HTTPS-ready.

VULNERABILITY 55 • HIGH

Weak Cryptography / Custom Crypto

WHAT IT IS

Sensitive data is protected with obsolete algorithms, weak modes, predictable randomness, or home-grown cryptography.

WHAT CAUSES IT

Developers implement encryption/signing themselves or use outdated primitives/configuration.

POTENTIAL IMPACT

Protected data may be recoverable or signatures/tokens forgeable.

HOW TO PREVENT / FIX IT

Use maintained cryptographic libraries and modern authenticated encryption/signature primitives with secure key management.

COPY-PASTE LLM PREVENTION PROMPT

> Audit encryption, hashing, signing, token generation, and random-number use. Remove custom cryptographic implementations and obsolete algorithms. Use maintained platform/library primitives with modern authenticated encryption where encryption is required, cryptographically secure randomness, and appropriate signing algorithms. Keep keys in protected server-side secret storage and define rotation.

VULNERABILITY 56 • HIGH

Insecure Randomness

WHAT IT IS

Security-sensitive values are predictable.

WHAT CAUSES IT

Normal pseudo-random functions are used for tokens, codes, session IDs, or reset links.

POTENTIAL IMPACT

Attackers may guess security tokens.

HOW TO PREVENT / FIX IT

Use the platform's cryptographically secure random generator with sufficient entropy.

COPY-PASTE LLM PREVENTION PROMPT

> Find generation of session IDs, API tokens, reset links, verification tokens, invitation codes, CSRF tokens, and other security-sensitive values. Replace non-cryptographic random functions, timestamps, counters, and

VULNERABILITY 57 • HIGH

Improper Certificate Validation

WHAT IT IS

The application accepts an untrusted TLS certificate when connecting to another service.

WHAT CAUSES IT

Certificate or hostname verification is disabled to make development/integration easier.

POTENTIAL IMPACT

A network attacker may impersonate the remote service.

HOW TO PREVENT / FIX IT

Keep certificate and hostname validation enabled; install proper CA certificates instead of disabling verification.

COPY-PASTE LLM PREVENTION PROMPT

```
> Search outbound HTTP/TLS clients for options that disable certificate or hostname verification. Remove insecure bypasses and use the platform's normal trust validation. For private services, configure the correct CA/certificate chain rather than accepting all certificates. Ensure production fails closed on TLS validation errors.
```

VULNERABILITY 58 • CRITICAL

Insecure Deserialization

WHAT IT IS

Untrusted serialized data is converted into application objects in a way that can trigger unsafe behavior.

WHAT CAUSES IT

Dangerous native serialization formats or polymorphic object creation are used on attacker-controlled data.

POTENTIAL IMPACT

Depending on the technology, attackers may alter application state or trigger code execution.

HOW TO PREVENT / FIX IT

Prefer simple data formats such as JSON with strict schemas; never deserialize untrusted native objects; allowlist types if unavoidable.

COPY-PASTE LLM PREVENTION PROMPT

```
> Audit all deserialization of cookies, request bodies, queue messages, cache entries, uploaded files, and integration payloads. Avoid native/object serialization formats for untrusted data. Prefer JSON or another simple format validated against a strict schema. Disable arbitrary type/polymorphic object creation and use explicit allowlists where a complex serializer is unavoidable.
```

VULNERABILITY 59 • HIGH

Prototype Pollution

WHAT IT IS

Attacker-controlled object keys modify JavaScript object prototypes or inherited properties.

WHAT CAUSES IT

Application logic may be corrupted and, in some contexts, security controls bypassed.

HOW TO PREVENT / FIX IT

Update vulnerable libraries, reject dangerous keys, use safe merge utilities, and avoid merging untrusted objects into security-sensitive configuration.

COPY-PASTE LLM PREVENTION PROMPT

> Audit JavaScript object merging, query parsing, configuration merging, and deep-assignment utilities. Update dependencies with known prototype-pollution issues. Reject dangerous property paths such as `__proto__`, `prototype`, and `constructor` where relevant, use safe maintained merge functions, and never merge untrusted objects into authorization or security configuration.

VULNERABILITY 60 • HIGH

Dependency With Known Vulnerabilities

WHAT IT IS

The application uses a third-party package with a publicly known security flaw.

WHAT CAUSES IT

Dependencies are outdated, unmaintained, pinned carelessly, or never scanned.

POTENTIAL IMPACT

Attackers may exploit the vulnerable component through the application.

HOW TO PREVENT / FIX IT

Maintain lockfiles, scan dependencies, patch promptly based on reachability/severity, and remove abandoned packages.

COPY-PASTE LLM PREVENTION PROMPT

> Generate an inventory of direct and transitive dependencies. Run the ecosystem's security audit/scanner and identify known vulnerabilities. Upgrade or replace vulnerable packages, prioritizing exploitable/reachable issues. Keep lockfiles committed, remove unused dependencies, and add automated dependency scanning to CI without blindly applying breaking upgrades.

VULNERABILITY 61 • CRITICAL

Malicious or Compromised Dependency

WHAT IT IS

A package or update contains intentionally harmful code or has been compromised upstream.

WHAT CAUSES IT

Developers install unverified packages, typosquatted names, or automatically accept risky updates.

POTENTIAL IMPACT

Build systems, secrets, production data, or users may be compromised.

HOW TO PREVENT / FIX IT

Minimize dependencies, verify package identity/provenance, lock versions, review major changes, restrict install scripts, and protect CI secrets.

COPY-PASTE LLM PREVENTION PROMPT

minimum secrets required.

VULNERABILITY 62 • HIGH

Dependency Confusion

WHAT IT IS

A build installs an attacker-controlled public package instead of the intended private package.

WHAT CAUSES IT

Private package names overlap with public registries and registry precedence/scoping is misconfigured.

POTENTIAL IMPACT

Malicious code may run during build or deployment.

HOW TO PREVENT / FIX IT

Use scoped private packages, explicit registry configuration, lockfiles, and controlled package publishing.

COPY-PASTE LLM PREVENTION PROMPT

> If private packages are used, audit package names and registry resolution. Use organization scopes/namespaces, explicitly configure the intended private registry for those scopes, commit lockfiles, and prevent fallback to untrusted public packages for private names. Add CI checks for unexpected registry sources.

VULNERABILITY 63 • MEDIUM

Exposed Source Maps / Build Artifacts

WHAT IT IS

Production publishes development artifacts that reveal source code, paths, or internal structure.

WHAT CAUSES IT

Source maps, debug bundles, backup files, or build metadata are deployed publicly.

POTENTIAL IMPACT

Attackers gain easier insight into application internals and sometimes secrets accidentally embedded in builds.

HOW TO PREVENT / FIX IT

Do not publish unnecessary source maps/artifacts publicly; if source maps are needed for monitoring, keep them private with the monitoring provider.

COPY-PASTE LLM PREVENTION PROMPT

> Audit production static assets for source maps, debug bundles, backup files, environment dumps, build manifests, and temporary artifacts. Remove public artifacts that are not required. If source maps are needed for error monitoring, upload them privately to the monitoring service and exclude them from public deployment. Verify no secrets exist in built assets.

VULNERABILITY 64 • CRITICAL

Exposed .env, Git, Backup, or Config Files

WHAT IT IS

Deployment copies hidden files, backups, repositories, or configuration into the served directory.

POTENTIAL IMPACT

Secrets, source code, or configuration may leak.

HOW TO PREVENT / FIX IT

Keep sensitive files outside the web root, block dotfiles/backups, and configure deployment allowlists.

COPY-PASTE LLM PREVENTION PROMPT

> Audit the deployed web root and static hosting configuration. Ensure .env files, .git directories, backups, database dumps, configuration files, logs, temporary files, and editor artifacts are never publicly served. Prefer an explicit deployment allowlist/build output rather than copying the entire repository. Add automated checks for common sensitive paths.

VULNERABILITY 65 • CRITICAL

Cloud / Database Service Role Key Exposure

WHAT IT IS

A highly privileged cloud or backend key is exposed to untrusted clients.

WHAT CAUSES IT

Service-role/admin credentials are placed in frontend code, mobile/browser configuration, logs, or public repositories.

POTENTIAL IMPACT

Attackers may bypass normal access controls and gain broad backend access.

HOW TO PREVENT / FIX IT

Keep service/admin keys server-only, rotate exposed keys, and use public/anon keys only with strong backend policies.

COPY-PASTE LLM PREVENTION PROMPT

> Search the repository, frontend bundle, logs, and deployment configuration for cloud admin keys, Supabase service-role keys, Firebase admin credentials, database admin tokens, and equivalent privileged credentials. Move them to protected server-only secret storage, rotate exposed keys immediately, and ensure browser clients use only intended public credentials backed by strict authorization policies.

VULNERABILITY 66 • CRITICAL

Database Row-Level Security Misconfiguration

WHAT IT IS

Database-level access policies allow users to read or modify rows they should not access.

WHAT CAUSES IT

RLS is disabled, policies use overly broad conditions, or write checks are missing.

POTENTIAL IMPACT

Cross-user or cross-tenant data exposure and modification may occur.

HOW TO PREVENT / FIX IT

Enable RLS where applicable, define explicit SELECT/INSERT/UPDATE/DELETE policies, use correct USING/WITH CHECK semantics, and test with multiple users.

COPY-PASTE LLM PREVENTION PROMPT

and privileged server roles.

VULNERABILITY 67 • CRITICAL

Multi-Tenant Data Isolation Failure

WHAT IT IS

One organization's users can access another organization's data.

WHAT CAUSES IT

Tenant identifiers are trusted from the client or not included consistently in authorization/database policies.

POTENTIAL IMPACT

A single customer may expose or modify another customer's data.

HOW TO PREVENT / FIX IT

Derive tenant membership from trusted server/session data and enforce tenant isolation at every data-access layer.

COPY-PASTE LLM PREVENTION PROMPT

> Audit all multi-tenant queries, routes, storage paths, caches, and background jobs. Never trust a tenant or organization ID merely because the client supplied it. Derive membership from the authenticated identity and enforce tenant constraints server-side and, where supported, in database policies. Add automated cross-tenant isolation tests.

VULNERABILITY 68 • CRITICAL

Webhook Signature Verification Missing

WHAT IT IS

The app accepts webhook requests without proving they came from the real provider.

WHAT CAUSES IT

Webhook signatures/secrets are ignored or verified incorrectly.

POTENTIAL IMPACT

Attackers may forge events such as payments, subscriptions, or account changes.

HOW TO PREVENT / FIX IT

Use the provider's official signature-verification method on the raw request body, reject invalid/timestamp-stale events, and protect against replay.

COPY-PASTE LLM PREVENTION PROMPT

> For every incoming webhook, use the provider's official SDK or documented cryptographic signature verification. Verify against the raw request body before parsing/mutating it, use the correct endpoint secret, enforce timestamp/replay protections where supported, and reject invalid events before any state change. Add tests for invalid signatures and duplicate events.

VULNERABILITY 69 • HIGH

Webhook Replay / Duplicate Processing

WHAT IT IS

Events are not deduplicated or state changes are not idempotent.

POTENTIAL IMPACT

Payments, credits, emails, or other actions may happen repeatedly.

HOW TO PREVENT / FIX IT

Store provider event IDs, process idempotently, use unique constraints/transactions, and safely acknowledge duplicates.

COPY-PASTE LLM PREVENTION PROMPT

> Make webhook processing idempotent. Store each provider event ID with a unique constraint before performing irreversible work, handle retries safely, and ensure duplicate delivery produces the same final state without repeating side effects. Use transactions where appropriate and test repeated delivery of the exact same valid event.

VULNERABILITY 70 • CRITICAL

Payment Amount / Price Tampering

WHAT IT IS

The backend trusts a price or total supplied by the browser.

WHAT CAUSES IT

Checkout requests accept client-calculated amounts, discounts, currency, or product prices.

POTENTIAL IMPACT

Attackers may pay less than intended or obtain incorrect entitlements.

HOW TO PREVENT / FIX IT

Look up product/price data server-side and create payment sessions from trusted records; verify completed events before granting access.

COPY-PASTE LLM PREVENTION PROMPT

> Audit checkout and payment creation. Never trust client-supplied prices, totals, discounts, currency, product names, or entitlement levels. Send only a product/price identifier from the client, retrieve authoritative pricing server-side, and create the payment with that trusted value. Grant access only after verifying the provider's completed payment event.

VULNERABILITY 71 • CRITICAL

Payment Webhook / Entitlement Logic Failure

WHAT IT IS

The app grants paid access based on an untrusted browser response or incomplete payment state.

WHAT CAUSES IT

Success pages, client callbacks, or loosely validated webhook fields directly activate subscriptions/products.

POTENTIAL IMPACT

Users may obtain paid features without a valid completed payment.

HOW TO PREVENT / FIX IT

Use verified server-side provider events as the source of truth and map them to known orders/customers.

COPY-PASTE LLM PREVENTION PROMPT

trusted records.

VULNERABILITY 72 • HIGH

Insecure API Key Authentication

WHAT IT IS

An API key is treated as a permanent password without sufficient protection or scope.

WHAT CAUSES IT

Keys are long-lived, broadly privileged, stored plaintext, sent in URLs, or cannot be revoked.

POTENTIAL IMPACT

A leaked key may provide broad persistent access.

HOW TO PREVENT / FIX IT

Generate high-entropy keys, show once, store hashed values, scope permissions, support rotation/revocation, and transmit only over HTTPS headers.

COPY-PASTE LLM PREVENTION PROMPT

> Harden application-issued API keys. Generate them with cryptographically secure randomness, display the secret only at creation, store a hashed representation server-side where practical, support scopes/least privilege, expiration or rotation, and revocation. Accept keys only over HTTPS in an authorization header, never in URLs, and log only a safe identifier/prefix.

VULNERABILITY 73 • HIGH

Missing API Authorization Scope Checks

WHAT IT IS

An authenticated API credential can perform actions outside its intended scope.

WHAT CAUSES IT

The server verifies that a token/key is valid but does not enforce scopes/permissions per endpoint.

POTENTIAL IMPACT

Integrations or leaked tokens gain excessive capabilities.

HOW TO PREVENT / FIX IT

Define least-privilege scopes and enforce required scopes server-side on every endpoint.

COPY-PASTE LLM PREVENTION PROMPT

> Define explicit API scopes/permissions for sensitive capabilities and map each endpoint/action to the minimum required scope. After authenticating the token or API key, enforce those scopes server-side before processing the request. Default to deny and add tests showing a valid low-scope credential cannot call higher-privilege endpoints.

VULNERABILITY 74 • CRITICAL

Insecure AI/LLM Tool Authorization

WHAT IT IS

The model is trusted to decide permissions, or tools run with broad service credentials without per-user authorization.

POTENTIAL IMPACT

Prompt manipulation or model mistakes may cause unauthorized data access or actions.

HOW TO PREVENT / FIX IT

Treat the model as untrusted orchestration. Enforce normal server-side authorization inside every tool/action and minimize tool privileges.

COPY-PASTE LLM PREVENTION PROMPT

> Audit every AI/LLM tool, function call, agent action, retrieval source, and backend capability. Treat model output as untrusted input. Before executing any tool, enforce the authenticated user's normal server-side authorization on the exact resource/action. Use least-privilege service credentials, explicit tool schemas/allowlists, and require human confirmation for irreversible high-impact actions.

VULNERABILITY 75 • HIGH

Prompt Injection Causing Unsafe Tool Use

WHAT IT IS

Untrusted content manipulates an AI feature into ignoring intended instructions or attempting unsafe actions.

WHAT CAUSES IT

The app gives an LLM access to tools/data and assumes prompts alone are a security boundary.

POTENTIAL IMPACT

The model may attempt unauthorized tool calls, disclose data it was given, or take unintended actions.

HOW TO PREVENT / FIX IT

Do not rely on system prompts for authorization. Isolate untrusted content, constrain tools, validate outputs, and enforce permissions outside the model.

COPY-PASTE LLM PREVENTION PROMPT

> Harden the AI feature against prompt injection. Treat all user content, webpages, documents, retrieved text, emails, and tool outputs as untrusted data, not instructions. Keep authorization and data-access decisions outside the model, expose only the minimum tools/data needed, validate tool arguments server-side, and require explicit confirmation before irreversible or high-impact actions.

Final launch reminder

A secure launch requires more than a clean AI review. Verify authentication and authorization with separate test accounts, test database and storage policies, confirm secrets are server-only, validate payment and webhook flows, test uploads and input handling, review dependencies, and confirm production configuration is hardened.

Use the prompts as defensive implementation instructions, then verify the resulting controls in the deployed application.